Group Membership in LLM Training Systems

Final Project for CS386C

Greyson Baker (gjb885), Devvrit Khatri (dd34332)

October 14, 2026

1 Introduction

Over the past decades, machine learning approaches to problem solving have demanded increasingly compute and data intensive systems. Despite computers themselves increasing in power over the period, industry standards and ballooning in the practical requirements of algorithms have made it extremely difficult to achieve acceptable results in reasonable timeframes on small scale devices. Especially with the advent of large language models (LLMs), it is not currently feasible to perform training of these models without dedicated clusters [Narayanan et al., 2021, Brown et al., 2020]. However, even within dedicated clusters, it is rare and highly difficult to be able to preassign all possible connections and configurations between individual devices and expect these to hold over a prolonged period of sustained and potentially variable workloads. This too made only more difficult that many devices which may individually be meaningfully different in capability and for system users which may desire to increase or decrease investment in the system and see that reflected in capability.

A number of simplifying assumptions have been made to easier design production systems. Differences in capability were smoothed over and the approach had largely been to accept a number of essentially identical worker nodes which can provide general compute toward the task at hand. These training jobs themselves would continue until either completion of the job or some failure was identified, in which case a job was merely moved to other workers blindly. With production systems now involving potentially huge numbers of devices, complex and variable jobs, and a desire to avoid long-term dedication of devices, these clusters become generally brittle to the real pressures imposed.

Moreover, training jobs are increasingly 'endless' [Gupta et al., 2023, Jindal et al., 2024]. Models may involve huge numbers of components or a constantly increasing quantity of data to review, at all times. This only adds to the criticality of being able to adjust configurations and have stable long-term solutions to usage standards. As failures may cause individual devices to become unavailable at any time, there is a distinct need to reconsider group membership and dynamics actively, such that optimal configuration can be reached, or at least approximated, throughout an entire training life cycle.

Thus, we must consider how to best address group membership and organization to address this increasingly complex set of issues endemic to modern LLM training. Given a potentially het-

erogeneous set of potentially faulty processors, an LLM training task set with uneven compute requirements, and variable objective expectations, we must ascertain how we can reliably and effectively establish groups and roles suitable to satisfying the tasks and objectives efficiently using the processor set.

2 Traditional Approaches

We will simplify slightly the analysis of these approaches and make the assumption that algorithms either provide protection against or there exist significant delays in new group formation due to joins. We will also assume that crashes are not extremely common, thus we will not consider the case where a group cyclically faults and reaches zero active processors. Should this actually happen, it would involve some catastrophic failure and some subsequent manual intervention. Since this is out of the realm of reasonability for our case, we will apply it to none of the algorithms nor schemes.

2.1 Group Membership Assumptions

In the scope of readings and lectures in the class, group membership is an active but distinctly fundamental step which seeks to use minimally intense communication to align knowledge about the group of processors available. It is subsequently expected that applications can leverage this information to otherwise communicate and establish tasks, workloads, or whatever else the group may require once the group is aligned. Moreover it seeks to handle some failures and other events, including handling crashes (or effective-crashes) of processors and join requests to a group.

The three algorithms considered make *who* is in a group, in terms of the processors, the prime consideration. This is, in its own right, a valuable step. Although there have been significant explorations into weakly-consistent membership [Das et al., 2002], anything qualifiable as as group must have some mechanism to determine which processors are in that group reliably.

2.2 Algorithmic Effectiveness

We will focus on Cristian's three major schemes for synchronous membership protocols: periodic broadcast, attendance list, and neighbor surveillance [Cristian, 1991].

2.2.1 Periodic Broadcast

At a basic level, the simplest approach often applies in the most circumstances, so perhaps it's critical to give them meaningful consideration even when alternatives are often better.

The periodic broadcast algorithm involves, on a timer period, every in-group processor (attempts to) send a message to every other group member. Then, after a delay, upon the failure of receipt of a message from a group member, begins, for the entire group, an atomic-broadcast request for membership update. Then, all living nodes respond to the request with their own *broadcasts*, at which point all nodes now have an accurate and complete list of all nodes in the now-group. Failures along the way will have been detected and replaced by their lack of inclusion in the last broadcasts. Joining for a relatively new processor involves merely requesting a new group via broadcast and finishing through every processors' broadcast just as before.

There's a clear benefit in simplicity here. The algorithm is relatively symmetric, can avoid a swathe of mid-reconfiguration failures, and is fairly quick if atomic broadcast times can be kept low. Taking Δ as the atomic broadcast time and π as the period of broadcasts, we can upper bound processor crash *correction* in $\pi + 2\Delta$. However, this particular algorithm inherently suffers issues with scales. Since every processor must engage in an atomic broadcast at least once per period, the message load on a network grows rapidly, roughly at $O(n^2)$, even there have been no faults.

It will be made clear later that although there can exist some identification of a leader by identifying the first node to call for a new group in the latest group formation, but it must be distinguished from a distinct leader. Especially in this algorithm, a leader is not persistent, is not held to hold any greater truth or authority, and does not identify a change in role from that time forward. It does, for this algorithm, not even guarantee that the 'leader' survived *most* of the formation process.

We can apply this algorithm clearly at some level to our case. Given a group of processors, they will be able to determine which processors they should treat as alive for the time being. So long as their followed some deterministic application of roles to the group, even if that application was anonymous to the aforementioned predispositions of devices, we could make a guarantee that roles would be assigned regularly throughout the group and that task progress could be made. Devices could be added and crashed without excessive harm, assuming there existed some mechanism for the redistribution of work according the roles aforementioned.

2.2.2 Attendance List

The attendance list scheme, taken as an evolution to the periodic broadcast, seeks to address a key message-efficiency complaint to the previous scheme with minimal disruption to the time required to detect and correct failures.

This scheme uses a single leader node passing an 'attendance list' through a tour of the known group members. At the period time, the leader sends a timestamped message to the first processor in the ordered membership list. This processor receives the list, appends its identifier and timestamp, and continues the process. If the original leader gets the message back with the expected identifiers in a reasonable timeframe, then the attendance list succeeds silently and group activity continues as normal. If any processor fails to receive the list in its reasonable timeframe, it will wait to see if any other processor has detected an error and then send out a broadcast to request a new group. Group reformation is essentially the same, thus we find a detection time for n processors in the group with δ datagram time bound between any two particular processors and a maximum clock variation of ε to yield a $\pi + n\delta + \varepsilon + 2\Delta$ detect and reconfigure time frame using the broadcast new group followed by broadcast all members setup. Joining follows the same request broadcast followed by presence broadcast as before.

Key, however, is that the expected number of messages is vastly lower. Indeed, a success merely requires n messages and a failing case does not-much-worse, still following an $O(n^2)$ atomic broadcast setup. This burden is massively lower. Presuming that $n\delta$ is not much greater than Δ, the detection latency is also not much greater.

This scheme does perturb the previous asterisk on role identification. There must be a clear leader node which starts the list each round. This, upon new group formation or reformation, will likely be the processor which calls for the new group. This processor *does* have a distinct role, though

not one selected due to any particular 'merit'.

It also satisfies our requirements a little better due to its scalability and definite shared ordering for a group scheme. This helps make some progress toward dynamic relationships and being able to hand more intensive clusters.

2.2.3 Neighbor Surveillance

In somewhat of a different direction, neighbor surveillance seeks to address scalability in low-failure latency times. Although we discussed that the previous method does well when $n\delta$ is not much greater than Δ, this isn't always a reasonable presumption. Indeed, we can almost guarantee that each node incurs some internal processing cost adding to δ and that longest network connection, thus a lower bound on Δ, is always at most the network total tour length, even if the tour is optimal.

Neighbor surveillance makes this improvement at some cost. It is now possible, with multiple unfortunately-timed crashes to have the group fail to detect processor failure for up to $k\pi + \delta + \Delta$, with another Δ to reconfigure, for k failures. Since the synchronization period is guaranteed to be larger, and likely will be much larger, than the processor-to-processor datagram bound. However, we gave in our assumption that this scenario is fairly unlikely. Instead, for a single failure, we reduce total reconfiguration time down to $\pi + \delta + \varepsilon + 2\Delta$, regardless of which node fails. Yet, the no-fault message count remains at n.

It does so by having each processor send a heartbeat on the period to its next node in the membership list. If a processor does not receive its predecessor's message in the time bounds, it must send a broadcast to call for a new group. Joining and reforming are otherwise similar to the other schemes.

Because of the ordering of nodes, this does require a definite membership ordering, however it does not require a distinct leader node in the way the previous scheme required.

The key advantage of increased scalability is clear. However, in LLM training workloads can involve significant dependencies on other nodes, so it may not be reasonable to incur the cost of a potential delay greater than $k\pi$ to address critical errors. The ability to scale to huge cluster groups may yet outcompete that possible delay given our reliability assumption. Now, nodes can expect better average awareness for single-fault systems, which may indeed allow for less time spent in a sub-par configuration overall.

3 Limitations

An advantage of the consideration of LLM training contexts that ought to be leveraged is that we can make requirements on the presence and count of various types of nodes. We can make requirements on the stability of communication, requirements on identifiers for particular devices, and limitations on the responsiveness of configuration changes too, since these are all near-guaranteed to be controllable by the system designer or otherwise expected for any real training system. Instead, focusing on the dynamic relationship of role, group, and resource selection can be left as the key challenges leading into the specific limitations of these algorithms.

Yet, there are a number of key limitations present in all of the algorithms above which must be addressed individually to modernize and adapt the system scheme to the new context. These limitations are difficult to address even through combinations of the above algorithms, so it is imperative that we consider the natures of these limitations such that more specific, or sophisticated, algorithms may be determined.

Thus there exists some value in disclaiming now that the previous approaches, although valuable independently for their capabilities and generality, there exists opportunity to grow into the niche relevant to these particular problems and these particular advantages.

3.1 Scalability

Despite consistent progress through the exploration of the previous algorithms in terms of scalability, there remains a huge quantity of progress yet to be made. Synchronous models demand strongly-consistent membership awareness. This helps applications be confident in what capabilities exist in a group, but it also means that any processor which appears to fail essentially stops application progress to reconfigure the entire group. In extremely large systems, although individual failures can be taken to be relatively rare, *some* failure occurring is relatively common. This is only intensified when schemes use larger or less stable networks, but still have strong requirements on connections due to, for example, dependencies between jobs.

We need a way to handle more processors and more powerful groups on average with less time spent on system reconfiguration tasks, even if the tradeoff involves batching some job work such that task size increases. Requiring $O(n^2)$ messages after *any* (detected) failure or after *any* rejoin is burdensome and worth additional scrutiny.

3.2 Heterogeneity

Moreover, networks and device capabilities are not guaranteed to be uniform nor ideal. Schemes like neighbor surveillance do fairly well at limiting the effect of non-ideal network topologies in that any particular device only need send and expect messages concerned with a very small portion of the network at large. However, at period time, the network does get a 'rush' of messages from the heartbeats. Attendance list avoids this rush by delaying out messages, but both of them are liable to fail if a particular portion of the network is just slow and attendance list is more likely to keep a moderately slowed component in the main group, since the first delayed receipt makes it, if no other processor fails, the new group leader. Thus, both schemes fail to consider how higher-variance networks benefit or suffer from particular slowed components. Both schemes fail to adapt the path as a whole, additionally, to take advantage of potentially faster or 'easier' network connections, but instead follow blindly a tour across the expected groups (whether by parts in surveillance or in whole by attendance list).

Yet even then device homogeneity suffers a different assumption as we move to examine task capabilities. Forming a single huge group from a set of processors does not take into consideration the capabilities of those devices. It merely stacks the processors into one large group, where subgroups and orchestrated advantages are not considered. Fundamentally, all of the previous schemes simply do not address the nature of individual devices being non-interchangable.

3.3 Configurability

Partially dependent on the basis of non-homogeneity, being able to actively and adaptively change configurations of groups easily is a valuable consideration to our use case. A user may have needs which change over the course of training and would prefer to avoid hard-stopping and reforming all the devices. Even more simply, they would prefer for the new configuration to be self-righting and adapt to whatever parameters are relevant automatically, rather than needing this parameters to be firm-coded.

Instead of viewing a group as a precursor to other generic work, we may instead observe the course of work and the demands upon the present processors to determine what the most effective group to *take* may be. Even if we expect to 'take' an entire set of processors to be dedicated for a particular group of tasks, determining how to distribute those tasks effectively between the group in a way that is most efficient requires some awareness and consideration of what any subgroup is fundamentally better at. Since doing so almost certainly requires some analysis of expectation of what the task will require and since many tasks are hard to declare to have definite workloads, there exist some variability and thus some opportunity for tradeoffs in handling of this work. Even in cases where this variability is insignificant, making determinations in advance on how groups should be expected to communicate and how algorithms should approach particular types of failure still opens an opportunity for configuration which users may wish to have control over beyond a blanket semantic and crash-rejoin behaviour.

4 Modern Approaches

4.1 Weakly-Consistent Membership in Cloud Systems

In contrast to the more synchronized models discussed earlier, many modern cloud and storage systems adopt weakly-consistent membership. These systems typically operate at much larger scales and under far less predictable network conditions, and they therefore accept that not all nodes will maintain an identical view of the group at the same moment. Instead, each node gradually learns about changes through lightweight, pairwise communication. The main goal becomes scalability and resilience rather than strict agreement at every instant.

A common example is the family of gossip-based membership protocols. Systems such as Amazon Dynamo [DeCandia et al., 2007], and Apache Cassandra [Lakshman and Malik, 2010] use a well-known protocol called Scalable Weakly-consistent Infection-style Membership (SWIM) [Das et al., 2002]. In SWIM, each node periodically "pings" another node and shares whatever membership information it has recently learned. Over time, information about failures and joins spreads through the cluster much like an epidemic. No single node is required to coordinate the view, and message cost grows slowly even as the system becomes very large. Failure detection is also probabilistic: a node is suspected of failure if enough pings go unanswered, but occasional false suspicions are tolerated because the protocol is designed to recover from them quickly.

Although weakly-consistent membership may seem too relaxed for ML training, it still serves an important purpose in the systems around it. Distributed training ultimately requires a strict, fully synchronized agreement on group membership, but protocols like SWIM are often used one layer below to determine which nodes *might* be alive and usable at all. This lightweight liveness

view scales well and lets the cluster react quickly to changes. Once a training job begins—or needs to recover from a failure—it performs its own stricter checks, such as PyTorch Elastic's rendezvous or NCCL's topology validation (which we'll discuss in our later sections), to form the final training group. This layering keeps ML systems both scalable and capable of enforcing the stronger guarantees required during actual training.

4.2 Role and Resource-Aware Membership in Orchestrators

Modern cluster orchestrators introduce another important shift in how group membership is handled: rather than treating all machines as interchangeable, they attach explicit roles and resource descriptions to each node. Systems such as Kubernetes [Kubernetes Authors, 2025], Slurm [Jette et al., 2002], Ray [Moritz et al., 2018], and Yarn [Vavilapalli et al., 2013] allow machines to advertise capabilities like available GPUs, memory, bandwidth, or special-purpose hardware. Scheduling decisions, and by extension, the groups of nodes assigned to a job, are then based not just on whether a node is alive, but whether it matches the role requirements of the task.

Kubernetes is a widely used example of this idea. Each node reports labels such as GPU type or hardware features, and jobs can request these roles using selectors or taints and tolerations. Ray extends this further by allowing tasks to declare the exact resources they need (e.g., "4 GPUs and 32GB of memory"), and the system forms groups of workers that satisfy these constraints. Slurm uses a similar idea with partitions and node features, ensuring that only nodes with the correct role or capability are admitted into the worker set for a particular job.

This model differs significantly from the classical membership algorithms discussed earlier, which primarily track which processors are alive and part of the group. In contrast, orchestrators determine *which subset of nodes is even eligible* to participate in a job. As ML workloads have become more specialized and heterogeneous, this capability-aware filtering has become essential; large training jobs often require nodes with consistent GPU types, compatible driver versions, or sufficient communication bandwidth. Thus, membership is no longer just a liveness question - it is a compatibility question.

These orchestrators do not, on their own, create the final synchronized training group. Instead, they provide the "candidate set" of nodes that match the required roles and resources. The distributed training framework (such as NCCL, Horovod, or PyTorch Elastic) then performs the stricter, tightly coordinated membership formation needed for actual collective operations. In this sense, role-aware orchestration complements the earlier weak membership layer: first determining which nodes *exist and satisfy the job's requirements*, and then handing off to the training framework to establish a strongly consistent group.

4.3 Elastic and Fault-Tolerant Membership in Distributed ML Training

Once a set of candidate nodes has been identified by the underlying cluster (through weak membership signals and through role and resource-aware orchestration), modern ML training frameworks must still create a synchronized group to actually run the training job. Unlike the cloud systems described earlier, distributed training requires all participants to agree on the exact group membership, world size, and communication topology before any collective operations can proceed. However, unlike the synchronous algorithms discussed in Section 2, these training groups must also

be able to survive failures and adjust to changes without restarting the entire job. This need has led to a set of "elastic" and fault-tolerant membership approaches.

Systems such as PyTorch Elastic PyTorch Developers [2025] and Tensorflow's Horovod Elastic [Sergeev and Del Balso, 2018] follow this idea by repeatedly checking which workers are currently available before forming a group. Workers announce that they are ready, and if some of them fail or disappear, the system simply waits until a stable enough set of workers exists to continue training. Once this stable set is found, the framework assigns ranks and builds the strict, synchronized group needed for collective operations. If a worker later crashes, these systems pause, reshape the group, and resume training with the remaining workers. Compared to the algorithms in Section 2, these elastic methods do not rely on fixed timing bounds or synchronized clocks; instead, they rely on retrying until a consistent view emerges. This makes them far more adaptable to large, real clusters while still providing the strong agreement required during the actual training steps.

4.4 Topology and Capability-Aware Collective Formation (NCCL, Gloo)

Even after a training framework determines which workers are alive and suitable for the job, the final step is to form a communication group that can actually perform collective operations such as all-reduce. Libraries like NVIDIA's NCCL [NVIDIA Corporation, 2025] and Facebook's Gloo [Facebook AI Research, 2025] take a more specialized approach to membership by explicitly considering the *topology* and *capabilities* of the participating nodes. Rather than treating all workers as interchangeable, these libraries try to form groups that minimize communication delays and avoid slow or incompatible links.

NCCL is the clearest example of this. Before collective operations begin, NCCL performs a discovery phase in which it inspects the GPU interconnect graph: NVLink, PCIe, or network fabric, and constructs rings or trees that avoid bottlenecks. A worker with a weaker link can significantly increase the effective δ in an all-reduce step, which in turn slows down the entire group. To prevent this, NCCL adjusts the topology or excludes incompatible devices to maintain predictable communication timing. Gloo performs a similar process in CPU-based environments, selecting transport mechanisms and routing paths based on available network interfaces.

This approach forms a contrast with the classical group membership algorithms discussed earlier. Those algorithms focus primarily on determining *who* is alive, whereas NCCL and Gloo also determine *how well* the members can communicate. In practice, this means that forming the final training group is not just a matter of synchronizing membership but also of selecting a configuration that keeps communication efficient. This layering - first detecting candidates, then filtering by resource roles, and finally arranging workers based on communication topology, allows distributed training systems to preserve both correctness and performance, even when devices differ in hardware or connectivity.

4.5 Emerging Research: Declarative or Capability-Based Group Formation

As distributed training systems continue to grow in scale and heterogeneity, a growing body of research focuses on making group formation more *declarative*. That is, allowing users or applications to express the requirements a training group must satisfy, and letting the system automatically assemble a suitable set of nodes. Rather than directly specifying which machines should participate, the user specifies constraints such as minimum GPU memory, acceptable network bandwidth,

consistency of device types, or even maximum tolerable communication delay. The system then searches for a group that satisfies these constraints, much like solving a small optimization or matching problem.

This idea extends the role-aware membership discussed earlier by treating group formation as a constraint satisfaction task. For example, if a training job cannot tolerate links slower than some effective δ, or requires a minimum number of identical accelerators, the system can filter out nodes that violate these conditions before any communication group is created. Some recent frameworks explore using capability descriptors - small summaries of each node's hardware, software, and communication properties, to automatically determine which subsets of nodes can form an efficient and stable training group. Others explore negotiation-based protocols, where potential members exchange capability summaries before agreeing on a final configuration, echoing the professor's point that group formation often requires deliberation even before a synchronized group exists.

Although still an active research area, these declarative and capability-based approaches point toward a more flexible future for group membership: one where systems can adapt to diverse hardware, dynamic scaling, and application-specific performance constraints while still achieving the strong agreement needed for collective training. By combining automatic capability evaluation with explicit training requirements, these methods aim to reduce manual configuration and improve both correctness and throughput in large, heterogeneous clusters.

5 Conclusion

Group membership has long been recognized as a foundational problem in distributed systems, and the classical synchronous schemes considered in this report illustrate both the clarity and the limits of that model. These algorithms give strong guarantees about agreement and timeliness, but they do so under assumptions that modern large-scale ML training systems rarely satisfy. In particular, today's clusters are highly heterogeneous, frequently reconfigured, and expected to support training jobs that may run for weeks without interruption. In such an environment, the cost of full-group reconfiguration after every failure or join event becomes impractical, even when the algorithms offer mathematically clean bounds such as $\pi + 2\Delta$ or $\pi + n\delta + \epsilon + 2\Delta$.

Modern training systems therefore adopt a more layered approach. At the lowest level, weakly-consistent membership protocols provide scalable awareness of which nodes may be alive, without enforcing strict synchrony across the cluster. Orchestrators then filter these nodes based on roles and hardware capabilities, narrowing the set of candidates to those that are actually suitable for a particular job. Distributed training frameworks build on top of this by forming tightly synchronized groups only when needed, often with the ability to reshape these groups elastically when failures occur. Finally, collective communication libraries incorporate topology and capability information to ensure that the selected group can communicate efficiently once training begins.

Seen together, these developments extend the spirit of classical membership protocols rather than discarding them. The central challenges identified in the synchronous setting - agreement under failures, the cost of reconfiguration, and the impact of communication delays, still appear in ML training systems, but in a form that requires more flexible and context-aware solutions. Emerging research on declarative and capability-based group formation reflects this shift, aiming to make group selection both more automatic and more aligned with application requirements.

As LLM training scales even further, the need to reason carefully about group membership will only increase. Whether through hybrid protocols, more expressive capability models, or new negotiation mechanisms, the goal remains the same as in the original synchronous setting: enabling a set of machines, however diverse, to operate as a coherent and reliable group. The tools and assumptions have changed, but the underlying problem is very much the same, and remains at the heart of making large-scale training possible.

References

Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, and et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.

Flaviu Cristian. Reaching agreement on processor group membership in synchronous distributed systems. *Distributed Computing*, 4:175 – 187, 1991. doi: 10.1007/BF01784719.

A. Das, I. Gupta, and A. Motivala. Swim: scalable weakly-consistent infection-style process group membership protocol. In *Proceedings International Conference on Dependable Systems and Networks*, pages 303–312, 2002. doi: 10.1109/DSN.2002.1028914.

Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Vosshall, and Werner Vogels. Dynamo: Amazon's Highly Available Key-value Store. In *SOSP*, pages 205–220. ACM, 2007.

Facebook AI Research. Gloo: Collective communications library. `https://github.com/facebookincubator/gloo`, 2025. Accessed: 2025-12-08.

Kritika Gupta, Louis Thérien, et al. Continual pre-training of large language models: How to (re)warm your model? *arXiv preprint arXiv:2308.04014*, 2023.

M Jette, C Dunlap, J Garlick, and M Grondona. Slurm: Simple linux utility for resource management. 07 2002. URL `https://www.osti.gov/biblio/15002962`.

Ishan Jindal et al. Balancing continuous pre-training and instruction fine-tuning for public large language models. *arXiv preprint arXiv:2410.10739*, 2024.

Kubernetes Authors. Labels and selectors; taints and tolerations, 2025. URL `https://kubernetes.io/docs/concepts/overview/working-with-objects/labels/`. This entry references the official Kubernetes documentation, which is subject to updates. The URL points to the 'Labels and Selectors' section, which is closely related to Node Labels. Taints and Tolerations documentation can be found in a related section within the official Kubernetes documentation.

Avinash Lakshman and Prashant Malik. Cassandra: a decentralized structured storage system. *SIGOPS Oper. Syst. Rev.*, 44(2):35–40, April 2010. ISSN 0163-5980. doi: 10.1145/1773912.1773922.

Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging ai applications, 2018. URL `https://arxiv.org/abs/1712.05889`.

Deepak Narayanan, Mohammad Shoeybi, Jared Casper, and et al. Efficient large-scale language model training on gpu clusters using megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.

NVIDIA Corporation. Nccl: Nvidia collective communication library. `https://developer.nvidia.com/nccl`, 2025.

PyTorch Developers. Torchelastic: Elastic and fault-tolerant training in pytorch. `https://docs.pytorch.org/docs/stable/distributed.elastic.html`, 2025. Accessed: 2025-12-08.

Alexander Sergeev and Mike Del Balso. Horovod: Fast and easy distributed deep learning in tensorflow. *arXiv preprint arXiv:1802.05799*, 2018.

Vinod Kumar Vavilapalli, Arun C. Murthy, Chris Douglas, Sharad Agarwal, Mahadev Konar, Robert Evans, Thomas Graves, Jason Lowe, Hitesh Shah, Siddharth Seth, Bikas Saha, Carlo Curino, Owen O'Malley, Sanjay R. Radia, Benjamin C. Reed, and Eric Baldeschwieler. Apache hadoop yarn: yet another resource negotiator. *Proceedings of the 4th annual Symposium on Cloud Computing*, 2013. URL `https://api.semanticscholar.org/CorpusID:11346384`.